asa assign1

jueun wi

2026-10-12

Problem 1

(a)

```r
set.seed(1)
n = 200
x = seq(0, 1, length.out = n)
y = sin(2*pi*x) + rnorm(n, sd = 0.15)

dat = data.frame(x = x, y = y)
write.csv(dat, here::here("data", "sim_data.csv"), row.names = FALSE)
```

(b)

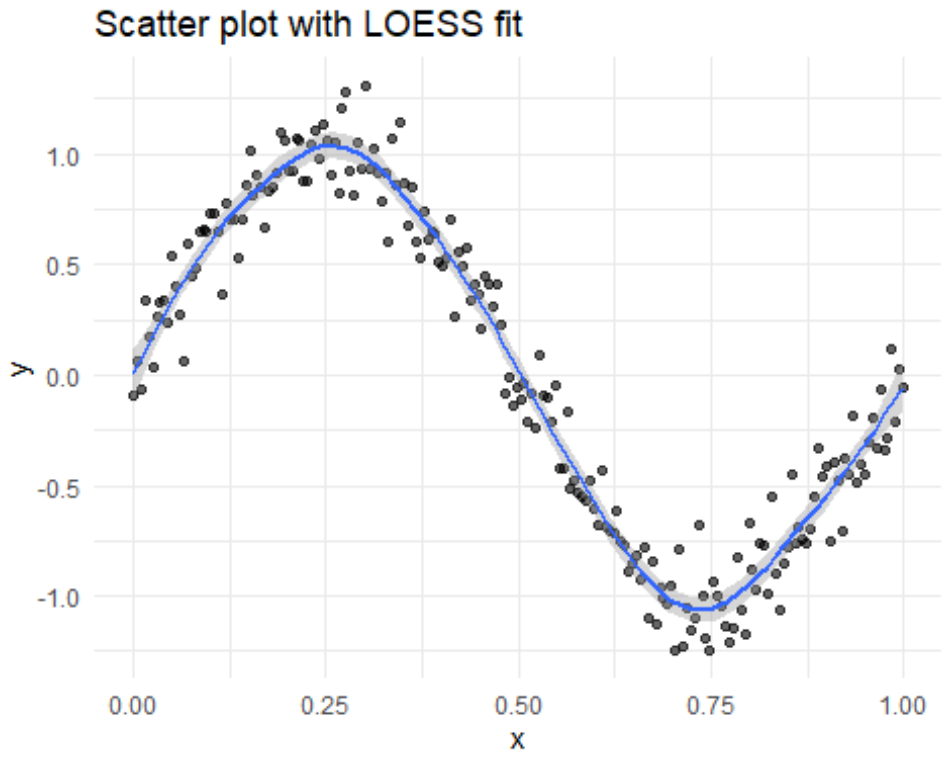
`R/plot_fit.R` 스크립트를 작성하여 데이터를 불러오고 시각화하였다. 데이터가 사인 함수 형태의 비선형 구조를 가지므로 직선 회귀(`lm`) 대신 국소 회귀인 `loess` 를 사용하였다.

```r
library(ggplot2)

dat = read.csv(here::here("data", "sim_data.csv"))

p = ggplot(dat, aes(x = x, y = y)) +
  geom_point(alpha = 0.6) +
  geom_smooth(method = "loess", formula = y ~ x, span = 0.3) +
  labs(title = "Scatter plot with LOESS fit") +
  theme_minimal()

p
```



(c)

`ggsave()`를 이용하여 (b)의 그림을 `plots/` 폴더에 저장하였다.

```
ggsave(here::here("plots", "scatter_loess.png"), plot = p,
       width = 6, height = 4, dpi = 300)
```

(d)

RStudio 에서 Git 버전 관리를 사용하여 commit 한 후 GitHub 에 업로드하였다.

GitHub 저장소 링크: https://github.com/jegh3211/asa-hw1

Problem 2

(a)

인접한 두 원소를 비교하여 순서가 맞지 않으면 교환하는 과정을 반복한다. 한 바퀴(pass) 동안 교환이 한 번도 일어나지 않으면 이미 정렬된 것이므로 조기 종료한다. `decreasing = TRUE` 이면 내림차순으로 정렬한다.

```r
bubble_sort = function(x, decreasing = FALSE) {
  n = length(x)
  if (n <= 1) return(x)

  for (i in 1:(n - 1)) {
    swapped = FALSE                # 이번 바퀴에 교환이 있었는지 기록

    for (j in 1:(n - i)) {         # 뒤쪽 i-1 개는 이미 정렬됨
      if (decreasing) {
        need_swap = x[j] < x[j + 1]
      } else {
        need_swap = x[j] > x[j + 1]
      }

      if (need_swap) {             # 두 원소 교환
        tmp = x[j]
        x[j] = x[j + 1]
        x[j + 1] = tmp
        swapped = TRUE
      }
    }

    if (!swapped) break            # 교환이 없으면 조기 종료
  }
  return(x)
}

set.seed(1)
x = runif(10)
x
```

```
##  [1] 0.26550866 0.37212390 0.57285336 0.90820779 0.20168193 0.89838968
##  [7] 0.94467527 0.66079779 0.62911404 0.06178627
```

```r
# 오름차순
bubble_sort(x)
```

```
##  [1] 0.06178627 0.20168193 0.26550866 0.37212390 0.57285336 0.62911404
##  [7] 0.66079779 0.89838968 0.90820779 0.94467527
```

```r
# 내림차순
bubble_sort(x, decreasing = TRUE)
```

```
##  [1] 0.94467527 0.90820779 0.89838968 0.66079779 0.62911404 0.57285336
##  [7] 0.37212390 0.26550866 0.20168193 0.06178627
```

```r
# R 내장 sort()와 결과 비교
all.equal(bubble_sort(x), sort(x))
```

```
## [1] TRUE
```

```r
all.equal(bubble_sort(x, decreasing = TRUE), sort(x, decreasing = TRUE))
```

```
## [1] TRUE
```

(b)

배열의 첫 번째 원소를 pivot 으로 선택하고, 나머지 원소를 pivot 보다 작은 그룹과 크거나 같은 그룹으로 분할한다. 각 그룹에 대해 재귀적으로 같은 과정을 반복하며, 그룹의 크기가 0 또는 1 이 되면 종료한다. `decreasing = TRUE` 이면 큰 원소를 왼쪽에 두어 내림차순으로 정렬한다.

```r
quick_sort = function(x, decreasing = FALSE) {
  if (length(x) <= 1) return(x)      # 크기가 0 또는 1 이면 종료

  pivot = x[1]                       # 첫 번째 원소를 pivot 으로
  rest = x[-1]                       # pivot 을 뺀 나머지

  if (decreasing) {
    left  = rest[rest >= pivot]      # 큰 원소는 왼쪽
    right = rest[rest < pivot]       # 작은 원소는 오른쪽
  } else {
    left  = rest[rest < pivot]       # 작은 원소는 왼쪽
    right = rest[rest >= pivot]      # 큰 원소는 오른쪽
  }

  # 왼쪽, 오른쪽을 재귀적으로 정렬한 뒤 합치기
  c(quick_sort(left, decreasing), pivot, quick_sort(right, decreasing))
}

set.seed(1)
x = runif(10)
x
```

```
##  [1] 0.26550866 0.37212390 0.57285336 0.90820779 0.20168193 0.89838968
##  [7] 0.94467527 0.66079779 0.62911404 0.06178627
```

```r
# 오름차순
quick_sort(x)
```

```
##  [1] 0.06178627 0.20168193 0.26550866 0.37212390 0.57285336 0.62911404
##  [7] 0.66079779 0.89838968 0.90820779 0.94467527
```

```
# 내림차순
quick_sort(x, decreasing = TRUE)
```

```
##  [1] 0.94467527 0.90820779 0.89838968 0.66079779 0.62911404 0.57285336
##  [7] 0.37212390 0.26550866 0.20168193 0.06178627
```

```
# R 내장 sort()와 결과 비교
all.equal(quick_sort(x), sort(x))
```

```
## [1] TRUE
```

```
all.equal(quick_sort(x, decreasing = TRUE), sort(x, decreasing = TRUE))
```

```
## [1] TRUE
```

Problem 3

(a)

전진, 후진, 중심차분을 method 인자로 선택할 수 있는 수치 미분 함수를 구현하였다. $f(x) = \cos(x) - x$의 해석적 도함수는 $f'(x) = -\sin(x) - 1$이며, $[0, 2\pi]$를 100 개의 점으로 균등 분할하여 비교하였다.

```
library(ggplot2)

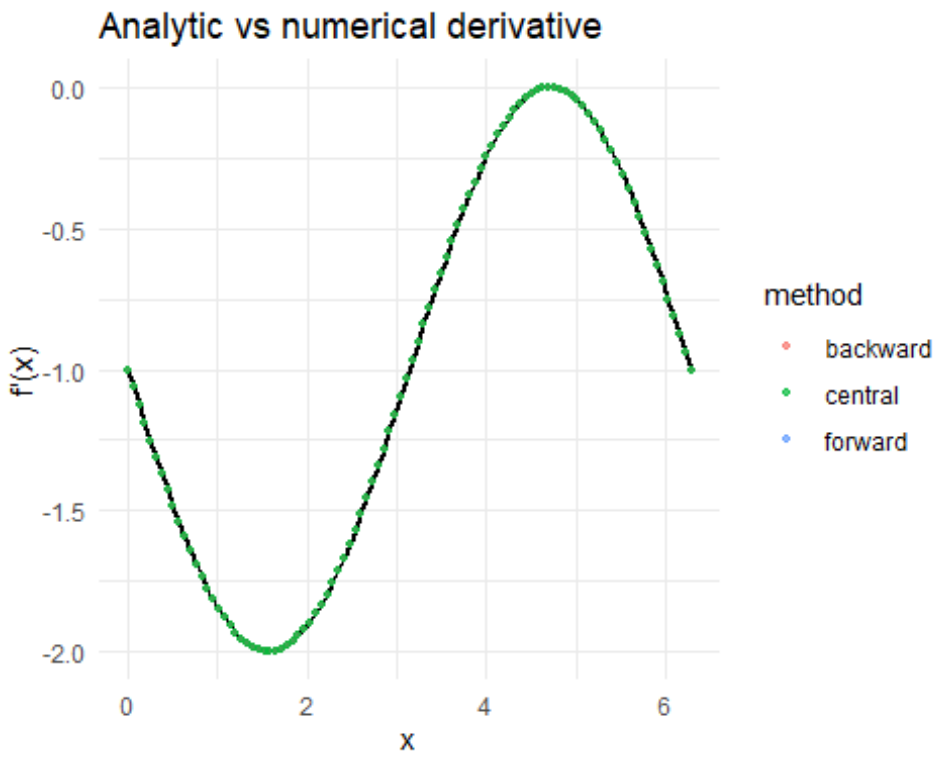
num_deriv = function(f, x, h = 1e-6, method = c("central", "forward", "backwa
rd")) {
  method = match.arg(method)
  if (method == "forward") {
    (f(x + h) - f(x)) / h
  } else if (method == "backward") {
    (f(x) - f(x - h)) / h
  } else {
    (f(x + h) - f(x - h)) / (2 * h)
  }
}

f = function(x) cos(x) - x
fprime = function(x) -sin(x) - 1          # 해석적 도함수

xs = seq(0, 2*pi, length.out = 100)
```

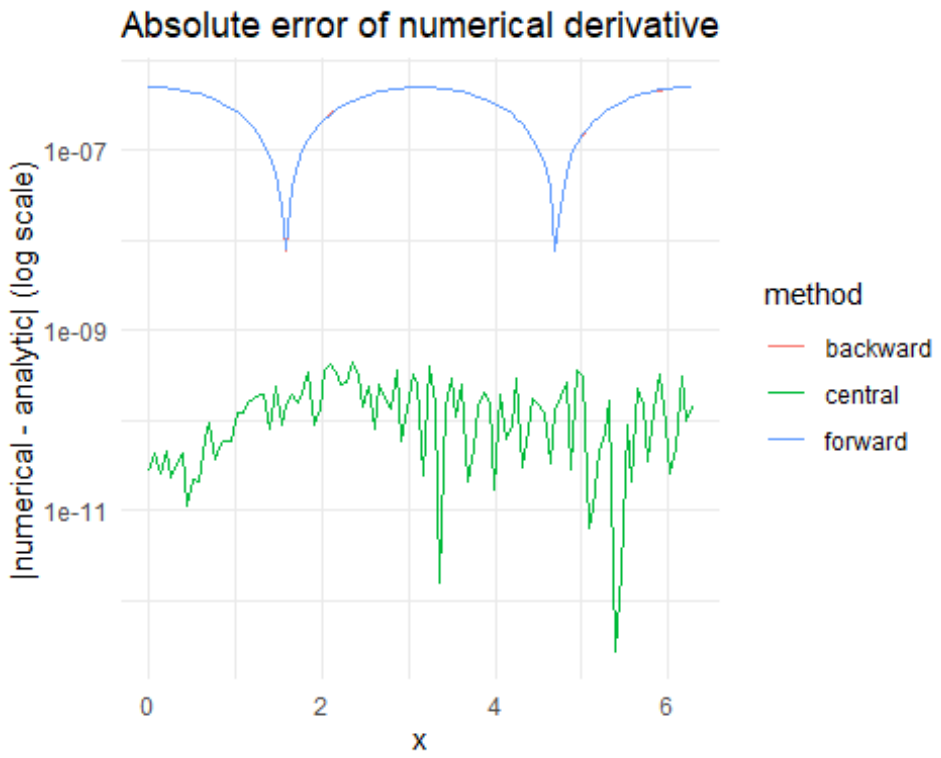
```r
# 세 방법으로 계산한 결과를 하나의 표로 모으기
res = rbind(
  data.frame(x = xs, method = "forward",  value = num_deriv(f, xs, method = "forward")),
  data.frame(x = xs, method = "backward", value = num_deriv(f, xs, method = "backward")),
  data.frame(x = xs, method = "central",  value = num_deriv(f, xs, method = "central"))
)
res$error = abs(res$value - fprime(res$x))  # 해석적 도함수와의 차이

# 그림 1: 해석적 도함수(검은 선)와 수치 미분(점)
ggplot() +
  geom_line(data = data.frame(x = xs, y = fprime(xs)), aes(x = x, y = y),
            linewidth = 1) +
  geom_point(data = res, aes(x = x, y = value, color = method),
             size = 1.2, alpha = 0.7) +
  labs(title = "Analytic vs numerical derivative",
       y = "f'(x)", color = "method") +
  theme_minimal()
```




```r
# 그림 2: 방법별 절대오차 (로그 스케일)
ggplot(res, aes(x = x, y = pmax(error, 1e-16), color = method)) +
  geom_line() +
```

```
  scale_y_log10() +
  labs(title = "Absolute error of numerical derivative",
       y = "|numerical - analytic| (log scale)") +
  theme_minimal()
```




```
# 방법별 평균 절대오차
aggregate(error ~ method, data = res, FUN = mean)
```

```
##     method        error
## 1 backward 3.201633e-07
## 2  central 1.418127e-10
## 3  forward 3.200912e-07
```

(b)

`fprime` 이 주어지면 해당 도함수를 사용하고, `NULL` 이면 (a)에서 구현한 `num_deriv()`의 중심차분(`method = "central"`)과 인자 `h` 를 이용하여 도함수를 근사한다. $|x_t - x_{t-1}| < \varepsilon$이면 수렴한 것으로 보고 종료한다.

```
newton = function(f, fprime = NULL, x0, maxiter = 100, h = 1e-6, epsilon = 1e
-10) {
  # 도함수가 없으면 중심차분 수치미분으로 대체
  if (is.null(fprime)) {
    fprime = function(x) num_deriv(f, x, h = h, method = "central")
```

```
  }

  x_old = x0
  for (t in 1:maxiter) {
    x_new = x_old - f(x_old) / fprime(x_old)   # Newton-Raphson 업데이트

    if (abs(x_new - x_old) < epsilon) {         # 수렴 판정
      return(list(root = x_new, iter = t, converged = TRUE))
    }
    x_old = x_new
  }

  warning("maxiter 안에 수렴하지 않았습니다.")
  list(root = x_old, iter = maxiter, converged = FALSE)
}
```

(c)

초깃값 $x_0 = 0.5$에서 수치미분 버전과 도함수 제공 버전을 각각 실행하였다.

```
f = function(x) cos(x) - x
fprime = function(x) -sin(x) - 1

# 수치미분 버전 (fprime 을 주지 않음)
res_num = newton(f, x0 = 0.5)
res_num
```

```
## $root
## [1] 0.7390851
##
## $iter
## [1] 5
##
## $converged
## [1] TRUE
```

```
# 도함수 제공 버전
res_exact = newton(f, fprime = fprime, x0 = 0.5)
res_exact
```

```
## $root
## [1] 0.7390851
##
## $iter
## [1] 5
##
```

```
## $converged
## [1] TRUE

# 두 결과 비교 및 f(root)가 0 에 가까운지 확인
print(c(numerical = res_num$root, exact = res_exact$root), digits = 12)

##      numerical          exact
## 0.739085133215 0.739085133215

f(res_num$root)

## [1] 0

f(res_exact$root)

## [1] 0
```

Problem 4

(a) Left Rectangle

```
left_rect = function(f, a, b, n) {
  h = (b - a) / n
  x = a + (0:(n - 1)) * h          # x_0, ..., x_{n-1}
  h * sum(f(x))
}
```

(b) Trapezoid

```
trapezoid = function(f, a, b, n) {
  h = (b - a) / n
  x = a + (0:n) * h                # x_0, ..., x_n
  fx = f(x)
  h / 2 * (fx[1] + 2 * sum(fx[-c(1, n + 1)]) + fx[n + 1])
}
```

(c) Simpson

```
simpson = function(f, a, b, n) {
  if (n %% 2 != 0) stop("Simpson 방법은 n 이 짝수여야 합니다.")
  h = (b - a) / n
  x = a + (0:n) * h
  fx = f(x)
  i = 0:n
  odd  = i[i %% 2 == 1]                    # 홀수 번째 점
  even = i[i %% 2 == 0 & i != 0 & i != n]  # 양 끝을 뺀 짝수 번째 점
  h / 3 * (fx[1] + 4 * sum(fx[odd + 1]) + 2 * sum(fx[even + 1]) + fx[n + 1])
}
```

(d)

$\int_0^{\pi} \sin(x)\,dx = 2$이다. $n = 100$으로 세 방법을 계산하였다.

```
res_d = c(
  left_rect = left_rect(sin, 0, pi, 100),
  trapezoid = trapezoid(sin, 0, pi, 100),
  simpson   = simpson(sin, 0, pi, 100)
)
print(res_d, digits = 12)
```

```
##     left_rect     trapezoid       simpson
## 1.99983550389 1.99983550389 2.00000001082
```

```
# 참값 2 와의 차이
print(abs(res_d - 2), digits = 4)
```

```
## left_rect trapezoid   simpson
## 1.645e-04 1.645e-04 1.082e-08
```

(e)

$n = 10,30,60,100,150,200$에 대해 참값 2 와의 절대오차를 계산하고 log-log 그래프로 비교하였다.

```
ns = c(10, 30, 60, 100, 150, 200)

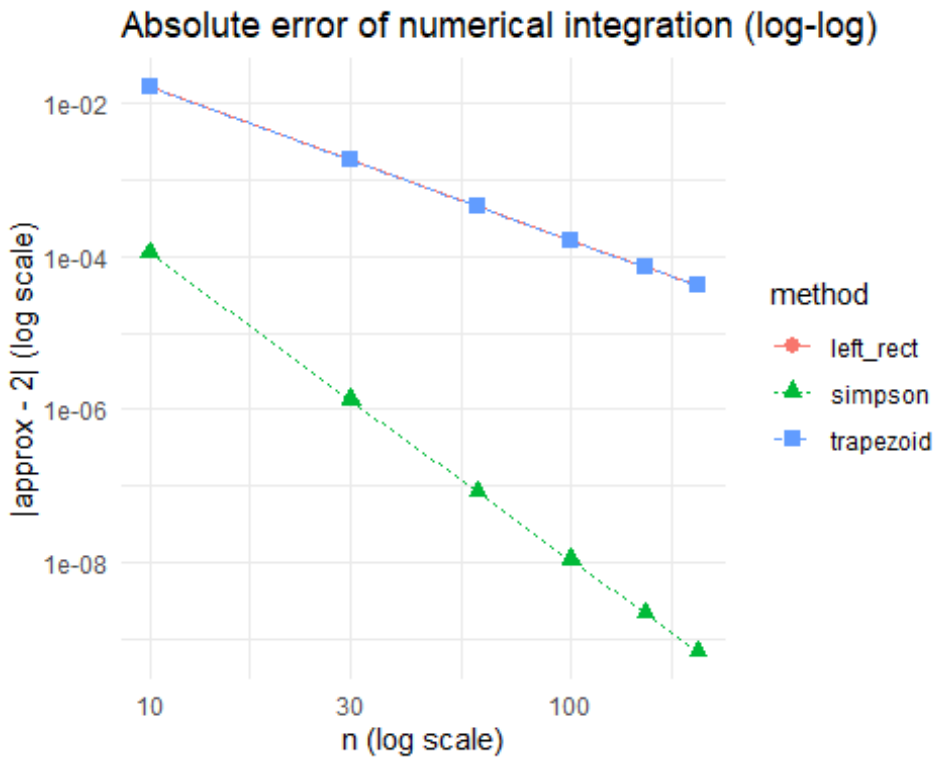
err_df = data.frame(
  n = rep(ns, 3),
  method = rep(c("left_rect", "trapezoid", "simpson"), each = length(ns)),
  error = c(
    abs(sapply(ns, function(n) left_rect(sin, 0, pi, n)) - 2),
    abs(sapply(ns, function(n) trapezoid(sin, 0, pi, n)) - 2),
    abs(sapply(ns, function(n) simpson(sin, 0, pi, n)) - 2)
  )
)

# 오차 표 (가로: 방법, 세로: n)
err_table = reshape(err_df, idvar = "n", timevar = "method", direction = "wid
e")
print(err_table, digits = 4)
```

```
##     n error.left_rect error.trapezoid error.simpson
## 1  10       1.648e-02       1.648e-02     1.095e-04
## 2  30       1.828e-03       1.828e-03     1.338e-06
## 3  60       4.569e-04       4.569e-04     8.354e-08
## 4 100       1.645e-04       1.645e-04     1.082e-08
```

```
## 5 150       7.311e-05       7.311e-05     2.138e-09
## 6 200       4.112e-05       4.112e-05     6.765e-10
```

```r
# log-log 그래프
ggplot(err_df, aes(x = n, y = error, color = method, shape = method, linetype
 = method)) +
  geom_line() +
  geom_point(size = 2.5) +
  scale_x_log10() +
  scale_y_log10() +
  labs(title = "Absolute error of numerical integration (log-log)",
       x = "n (log scale)", y = "|approx - 2| (log scale)") +
  theme_minimal()
```




```r
# log(error) ~ log(n)의 기울기 = 수렴 속도
sapply(split(err_df, err_df$method),
       function(d) coef(lm(log(error) ~ log(n), data = d))[2])
```

```
## left_rect.log(n)   simpson.log(n) trapezoid.log(n)
##        -2.000498        -4.003566        -2.000498
```

Problem 5

(a)

`chol()`은 $A = U^\top U$인 상삼각행렬 U를 반환하므로, $L = U^\top$로 구한다.

```
A = matrix(c(4,2,2,2,5,1,2,1,3), 3)
A
```

```
##      [,1] [,2] [,3]
## [1,]    4    2    2
## [2,]    2    5    1
## [3,]    2    1    3
```

```
L = t(chol(A))
L
```

```
##      [,1] [,2]     [,3]
## [1,]    2    0 0.000000
## [2,]    1    2 0.000000
## [3,]    1    0 1.414214
```

```
L %*% t(L)                  # A 와 같아야 함
```

```
##      [,1] [,2] [,3]
## [1,]    4    2    2
## [2,]    2    5    1
## [3,]    2    1    3
```

```
all.equal(L %*% t(L), A)
```

```
## [1] TRUE
```

(b)

$z_i = \frac{1}{l_{ii}}\left(b_i - \sum_{j=1}^{i-1} l_{ij}\, z_j\right)$를 $i = 1, \ldots, n$ 순서로 계산한다.

```
forward = function(L, b) {
  n = length(b)
  z = numeric(n)
  for (i in 1:n) {
    s = 0
    if (i > 1) s = sum(L[i, 1:(i - 1)] * z[1:(i - 1)])
    z[i] = (b[i] - s) / L[i, i]
  }
  z
}

b = c(1, -2, 3)
```

```r
z = forward(L, b)
z
```

```
## [1]  0.500000 -1.250000  1.767767
```

```r
forwardsolve(L, b)          # R 내장 함수와 비교
```

```
## [1]  0.500000 -1.250000  1.767767
```

```r
all.equal(z, forwardsolve(L, b))
```

```
## [1] TRUE
```

(c)

$x_i = \frac{1}{l_{ii}}\left(z_i - \sum_{j=i+1}^{n} l_{ji}\, x_j\right)$이다. x_i를 구하려면 $x_{i+1}, \ldots, x_n$이 필요하므로 $i = n, \ldots, 1$의 역순으로 계산한다. R 의 내장 함수는 `backsolve()`이며, 상삼각행렬 $L^\top$을 입력한다.

```r
backward = function(L, z) {
  n = length(z)
  x = numeric(n)
  for (i in n:1) {            # 아래에서부터 거꾸로
    s = 0
    if (i < n) s = sum(L[(i + 1):n, i] * x[(i + 1):n])
    x[i] = (z[i] - s) / L[i, i]
  }
  x
}

x = backward(L, z)
x
```

```
## [1] -0.0625 -0.6250  1.2500
```

```r
backsolve(t(L), z)          # R 내장 함수와 비교
```

```
## [1] -0.0625 -0.6250  1.2500
```

```r
all.equal(x, backsolve(t(L), z))
```

```
## [1] TRUE
```

(d)

$Lz = b$를 `forward()`로, $L^\top x = z$를 `backward()`로 풀어 $Ax = b$의 해를 구하고 `solve(A, b)`와 비교하였다.

```r
b = c(1, -2, 3)
z = forward(L, b)
x = backward(L, z)
x
```

```
## [1] -0.0625 -0.6250  1.2500
```

```r
solve(A, b)
```

```
## [1] -0.0625 -0.6250  1.2500
```

```r
all.equal(x, solve(A, b))
```

```
## [1] TRUE
```

Problem 6

(a)

두 점 사이의 Gaussian kernel 값을 계산하는 함수와, 이를 이용해 커널 행렬을 만드는 함수를 작성하였다.

```r
gauss_kernel = function(x, xprime, rho = 1) {
  exp(-rho * sum((x - xprime)^2))
}

# 행렬 X1 의 각 행과 X2 의 각 행 사이의 커널 값으로 행렬 만들기
kernel_matrix = function(X1, X2, rho = 1) {
  X1 = as.matrix(X1)
  X2 = as.matrix(X2)
  K = matrix(0, nrow(X1), nrow(X2))
  for (i in 1:nrow(X1)) {
    for (j in 1:nrow(X2)) {
      K[i, j] = gauss_kernel(X1[i, ], X2[j, ], rho)
    }
  }
  K
}

gauss_kernel(0, 0)     # 같은 점: 1
```

```
## [1] 1
```

```r
gauss_kernel(0, 1)     # exp(-1)
```

```
## [1] 0.3678794
```

(b)

$\hat{\alpha} = (K + \lambda I_n)^{-1} y$를 계산하고, 새로운 데이터 예측과 시각화에 필요한 학습 데이터, $\hat{\alpha}$, ρ, λ를 저장한 뒤 class 를 `krr` 로 지정하였다.

```
krr = function(X, y, rho = 1, lambda = 1e-4) {
  X = as.matrix(X)
  n = nrow(X)
  K = kernel_matrix(X, X, rho)
  alpha = solve(K + lambda * diag(n), y)   # (K + lambda I)^{-1} y

  obj = list(alpha = as.vector(alpha), X = X, y = y,
             rho = rho, lambda = lambda,
             fitted = as.vector(K %*% alpha))
  class(obj) = "krr"                       # 이름표 붙이기
  obj
}
```

(c)

$\hat{f}(x) = k(x, X)^\top \hat{\alpha}$를 계산하는 `predict.krr()`을 작성하였다.

```
predict.krr = function(object, newdata, ...) {
  newdata = as.matrix(newdata)
  K_new = kernel_matrix(newdata, object$X, object$rho)  # 새 점 x 학습 점
  as.vector(K_new %*% object$alpha)
}
```

(d)

산점도와 추정 함수 $\hat{f}(x)$를 그리는 `plot.krr()`을 작성하였다. 참 함수가 주어지면 함께 표시한다.

```
plot.krr = function(x, ftrue = NULL, ngrid = 300, ...) {
  if (ncol(x$X) != 1) stop("1 차원 입력만 시각화할 수 있습니다.")

  grid = seq(min(x$X), max(x$X), length.out = ngrid)
  fhat = predict(x, grid)

  plot(x$X[, 1], x$y, pch = 16, col = "gray50",
       xlab = "x", ylab = "y",
       main = sprintf("KRR fit (rho = %g, lambda = %g)", x$rho, x$lambda),
 ...)
  lines(grid, fhat, col = "red", lwd = 2)
```

```
  if (!is.null(ftrue)) {
    lines(grid, ftrue(grid), col = "blue", lwd = 2, lty = 2)
    legend("bottomleft", c("data", "KRR fit", "true f"),
           col = c("gray50", "red", "blue"), pch = c(16, NA, NA),
           lty = c(NA, 1, 2), lwd = c(NA, 2, 2), bty = "n")
  } else {
    legend("bottomleft", c("data", "KRR fit"),
           col = c("gray50", "red"), pch = c(16, NA),
           lty = c(NA, 1), lwd = c(NA, 2), bty = "n")
  }
  invisible(x)
}
```

(e)

```
set.seed(1)
n = 150
X = matrix(runif(n, -1, 1), ncol = 1)
ftrue = function(x) sin(2*pi*x) + 0.5*cos(4*pi*x)
y = ftrue(X[,1]) + rnorm(n, sd = 0.1)

# 기본값 (rho = 1, lambda = 1e-4)으로 적합
fit1 = krr(X, y)
class(fit1)

## [1] "krr"

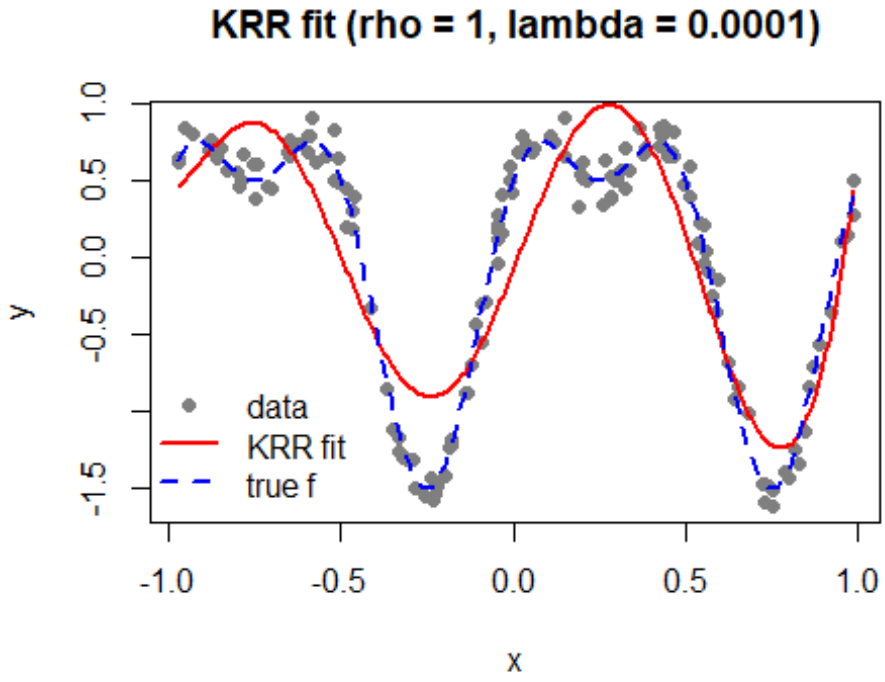
# 새로운 점에서의 예측
x_new = c(-0.5, 0, 0.5)
predict(fit1, x_new)

## [1]  0.008939831 -0.075240928  0.147330941

ftrue(x_new)

## [1] 0.5 0.5 0.5

plot(fit1, ftrue = ftrue)
```

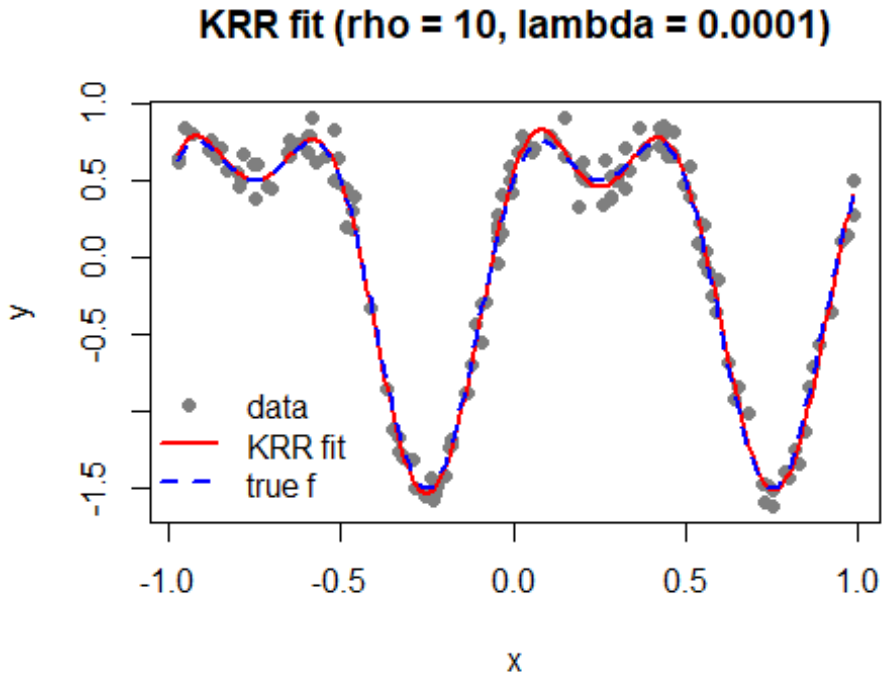


기본값 $\rho = 1$에서는 커널의 폭이 데이터 범위 $[-1,1]$에 비해 너무 넓어 빠르게 진동하는 참 함수를 충분히 따라가지 못한다(과소적합). ρ를 키워 커널 폭을 좁히면 적합이 크게 개선된다.

```
fit10 = krr(X, y, rho = 10)
predict(fit10, x_new)

## [1] 0.5041309 0.5455797 0.5164949

plot(fit10, ftrue = ftrue)
```




```
# 참 함수와의 RMSE 비교
grid = seq(-1, 1, length.out = 300)
c(rho1  = sqrt(mean((predict(fit1,  grid) - ftrue(grid))^2)),
  rho10 = sqrt(mean((predict(fit10, grid) - ftrue(grid))^2)))
```

```
##       rho1      rho10
## 0.31563420 0.03234759
```